

北京邮电大学

《程序设计实验》概要设计文档



题目：基于 Agent 技术的形式语言与自动机辅助学习游戏
《GodSav》

班 级	2022219107
学 号	2022211799 2022211733 2022211740 2022211738
姓 名	王艺栋 王殿云 池瀚 王博远
指导教师	刘瑞芳

2024 年 4 月

1、系统概述

1.1、系统简介

本项目服务于北京邮电大学《形式语言与自动机》课程教学，以 RPG（角色扮演）游戏的形式带领玩家一步步深入探索神奇的魔法世界，在完成任务与战斗的过程中逐渐学习形式语言与自动机的核心概念与算法。不同于传统游戏基于规则的 NPC 设计方法与固定的剧情线路，我们的项目通过人物设计与 ChatGPT 接入，为游戏引入了由 ChatGPT 自主控制的 Agent（智能体），为玩家提供了形式语言与自动机的全自动答疑，与更加丰富的交互体验，让玩家在新奇且沉浸的体验中，感受形式语言与自动机知识的美妙。本项目主要使用 web 开发技术，让玩家可以通过浏览器访问我们的游戏，提供全平台浏览器支持（Windows，MacOS，Android 与 IOS），跨越了平台壁垒，为更多玩家提供服务。

项目目前已由 GitHub Page 托管发布，欢迎您来体验！

游戏：

<https://wangdianyun.github.io/GodSay/>

展示：

<https://gamma.app/docs/GodSay-tv028highfk56sk>

本系统设计的主要创新点有：

1. 寓教于乐的教学模式

不同于传统教学辅助工具的枯燥乏味，本系统引入 ARPG 游戏模式，帮助学生在游戏中学习新知识，在增强学生学习自主性的同时降低了知识学习过程中的枯燥乏味。

2.LLM 赋能 Agent 自主答疑

大语言模型自主决策，让游戏中的非玩家角色拥有自主探索环境、不设限回复的能力。突破了传统游戏中 NPC 交互死板的局限，实现了游戏角色的智能交互，增加了游戏的沉浸感。并且采用模块化设计，可以灵活地定制和扩展角色的能力，便于维护和管理。

3. 内嵌式形式语言与自动机解密插件

知识只有在应用中才能掌握扎实，通过开发形式语言与自动机解密插件，我们为玩家们提供了在游戏中交互式解题的方式，并在游戏中加入了大量解谜元素。谜题即形式语言与自动机的题目。玩家可以通过构造自己的 DFA、NFA、CFG、REG 和 PDA 来获得武器、解除禁制、学习魔法。

在游戏中，我们为玩家们设计了一个自动机可视化构造模块，它主要负责给用户提供直观友好的自动机状态和转换关系编辑页面，帮助用户直观理解自动机的运行和状态转移，并在解决谜题的同时学习知识。它的主要功能包括：绘制状态图、编辑状态和转换、设置初始和终止状态、输入文法、测试自动机接受的字符串。

1.2、系统目标

1.2.1 功能目标：

本软件项目《GodSay》希望实现一个在线角色扮演游戏，用于辅助形式语言与自动机教学。玩家输入操作通过浏览器进行，系统经过服务器处理后反馈游戏状态更新至玩家界面。功能包括：

- 教学互动

实现游戏内任务和挑战，自动引导玩家学习形式语言与自动机的核心概念。

提供循序渐进的任务设计，从基础到复杂，帮助玩家逐步掌握知识点。

- **Agent**
利用 ChatGPT-3.5 驱动的 NPC，提供实时问题解答与自主交互功能。
增强玩家沉浸感和体验感，帮助玩家了解游戏世界观及其背景。
- **进度跟踪**
记录玩家的学习进度和成绩，生成详细的学习报告。
提供教师和学生参考，支持导出和分享学习报告。
- **多用户支持**
支持至少 500 名并发用户操作，确保系统在高负载情况下的稳定响应。
采用负载均衡和性能优化技术，保障系统的流畅运行。
- **核心功能：自动机构建**
用户可以在界面内进行拖拽操作，构建能够识别给定形式语言的自动机。
提供状态图和箭头工具，实时判断用户构建的自动机的正确性并推动剧情发展。
- **错误提示与反馈**
根据玩家解答结果，输出正确答案和错误提示。
提供用户掌握程度等相关信息，帮助用户更好地学习和理解自动机概念与构建过程。

软件的核心功能是让用户根据给定的形式语言，在界面内进行拖拽操作，构建可以识别该语言的自动机。用户可以拖拽状态图和箭头来设计自动机的结构，判断用户构建的自动机的正确性来推动剧情进行。

形式语言与自动机构造页面可以根据玩家解答的结果输出正确答案、错误提示以及用户的掌握程度等相关信息，以帮助用户更好地学习和理解自动机的概念和构建过程。

通过这种交互式的方式，用户可以直观地了解自动机的工作原理，快速掌握自动机的构建方法，并及时纠正错误，提高学习效率。这种软件设计不仅可以为用户提供学习自动机的工具，还可以通过实时反馈和指导，帮助用户在学习过程中加深对自动机理论的理解和掌握。

1.2.2 性能目标：

- **高并发性能**
系统需处理大量用户同时访问，确保高负载下的稳定性。
优化并发访问，包括线程管理、资源调度和数据库连接池管理等。
- **高可用性**
确保系统在故障或意外情况下依然正常运行，避免单点故障。
采用负载均衡、故障转移和容灾备份技术，保证系统高可用性。
- **响应时间优化**
确保用户操作的快速响应，避免长时间等待或卡顿。
进行性能优化，包括代码优化、数据库查询优化、缓存策略和异步处理等，提升系统响应速度和用户体验。
- **可扩展性**
系统需具备良好的水平和垂直扩展能力，满足未来的用户量和业务需求。
设计具有可扩展性的系统架构和数据库结构，采用云计算等技术实现按需扩展。

1.3、系统运行环境

我们的系统使用服务器托管，用户端使用 HTTP 或 HTTPS 协议向服务器发起请求并拉取游戏内容，服务器返回游戏的逻辑与数据，具体信息如下：

1.3.1 服务器硬件平台

系统的服务器端运行在支持 Ubuntu (≥20.04) 或 CentOS (≥7-1511) 系统的服务器硬件平台 (AMD64、Intel 64 与 Intel x86 为主) 上，服务器用于托管 Web 页面与资源，并供客户端访问。出于性能考虑，每台服务器硬件平台至少需要拥有 64GB 的内存与 128GB 的储存空

间，并通过 CDN 处理高并发请求。出于成本考虑，本项目选择 GitHub 的页面托管服务来分发页面，由 GitHub 自身的服务器和基础设施托管。GitHub 的服务器位于多个数据中心，具有高可用性、稳定性和性能。并且 GitHub Pages 使用 CDN 来加速内容交付，提供全球范围内快速访问网站的能力，符合我们的要求。

1.3.2 服务器操作系统

出于稳定性和开源特性，系统的服务器端主要采用 Linux 操作系统，考虑性能和特性，具体主要包括 Ubuntu (≥ 20.04) 或 CentOS ($\geq 7-1511$)，为用户提供可靠安全的服务。

1.3.3 客户端硬件平台

客户端硬件平台要求主要集中于图形处理与网络请求，本游戏采用 2D 像素风设计，无 3D 渲染需求，因此客户端的硬件需要能荷载 60 帧 720P 的简单图形处理，并同步处理交互与音频等信息，要求硬件具有 4G 及以上的内存。网络请求上，出于 ChatGPT 的交互需求，要求客户端能够正常与 <https://api.openai.com/v1/chat/completions> 完成通信。

1.3.4 客户端操作系统

客户端操作系统无特殊限制，仅需包含图形化界面支持。由于本项目用 web 页面交互，并针对包括 Chrome、Edge、Firefox、Safari 在内的浏览器优化，因此理论上用户可以使用 Windows、MacOS、Android、IOS、Ubuntu 在内的系统访问我们的系统。

1.3.6 数据库系统

本项目使用了关系型数据库 MySQL 完成用户信息的查询与修改，并能保存不同用户的游戏进度，帮助用户循序渐进地学习知识。

1.3.7 编程平台

前端开发主要使用 HTML、CSS、JavaScript 进行，构建网页结构、样式和交互，并且使用 RPGmakerMZ web 游戏开发框架完成形式语言与自动机答题页面的设计，使用 Node.js 完成项目的管理与测试开发。

后端开发较为轻量级，因此主要使用 python 结合 MySQL 进行开发，使用 mysql-connector-python 框架完成功能设计与开发。

1.3.8 网络协议

本项目中主要使用到的网络协议包括 HTTP 和 HTTPS 协议，以在服务器端与客户端之间通信。

网络通信操作在本项目中主要负责：

1. 从服务器拉取游戏页面与资源，供玩家游玩；
2. 从服务器拉取形式语言与自动机解题页并嵌入游戏页面；
3. 与 <https://api.openai.com/v1/chat/completions> 通信，调取 ChatGPT 的 API 以完成 Agent 的构建；

1.4、开发环境

IDE:

IntelliJ IDEA 2024.1 (Ultimate Edition)

PyCharm 2024.1.1 (Professional Edition)

RPG Maker MZ

开发语言：

HTML(Living Standard)

JavaScript (ES2023)

Python (3.11.8)

SQL(适用 8.0.36 MySQL Community Server)

包管理：

Node.js (18.20.0)

Conda (23.7.4)

项目管理工具：

Git(2.42.0.windows.2)

库：

Python 库

mysql-connector-python: 8.4.0

flask: 3.0.3

Node 包

bootstrap: 4.4.1,

cfgrammar-tool: 1.0.0,

d3: 5.15.0,

react: 16.12.0,

react-bootstrap: 1.0.0-beta.16,

react-contextmenu: 2.13.0,

react-d3-graph: 2.3.0,

react-dom: 16.12.0,

react-files: 2.4.8,

react-scripts: 3.3.1,

react-sidebar: 3.0.2,

react-tooltip: 4.2.9,

reactjs-popup: 1.5.0,

uuid: 8.0.0,

vis-network: 7.3.3

测试环境

服务器 Ubuntu20.04

客户端 Windows 11 家庭中文版：

Google Chrome 125.0.6422.60 (正式版本) (64 位)

Microsoft Edge 125.0.2535.51 (正式版本) (64 位)

Firefox Browser 126.0 (64 位)

客户端 IOS 17.4.1

Safari 4.0

1.5、关键知识点

序号	知识点 1	关系	知识点 2
1	形式语言与自动机	包含	DFA 的构造与识别
2	形式语言与自动机	包含	NFA 的构造与识别
3	形式语言与自动机	包含	CFG 的构造与识别
4	形式语言与自动机	包含	PDA 的构造与识别
5	形式语言与自动机	包含	REG 的构造与识别
6	形式语言与自动机	包含	语言、文法基础知识
7	前端开发	包含	HTML 语言
8	前端开发	包含	CSS 语言
9	前端开发	包含	JavaScript 语言
10	前端开发	包含	React 前端开发框架
11	前端开发	包含	Node.js 使用方法
12	前端开发	包含	Webpack 打包方法
13	后端开发	包含	Python 语言
14	后端开发	包含	MySQL 数据库
15	后端开发	包含	SQL 语言
16	后端开发	包含	mysql-connector-python 库使用
17	后端开发	包含	Flask 库使用
18	网络通信协议	包含	HTTP
19	网络通信协议	包含	HTTPS

20	网络通信协议	包含	GET 方法
21	API 接口协议	包含	ChatGPT API 接口
22	Prompt 工程	包含	世界观 Prompt 设计
23	Prompt 工程	包含	角色人格 Prompt 设计

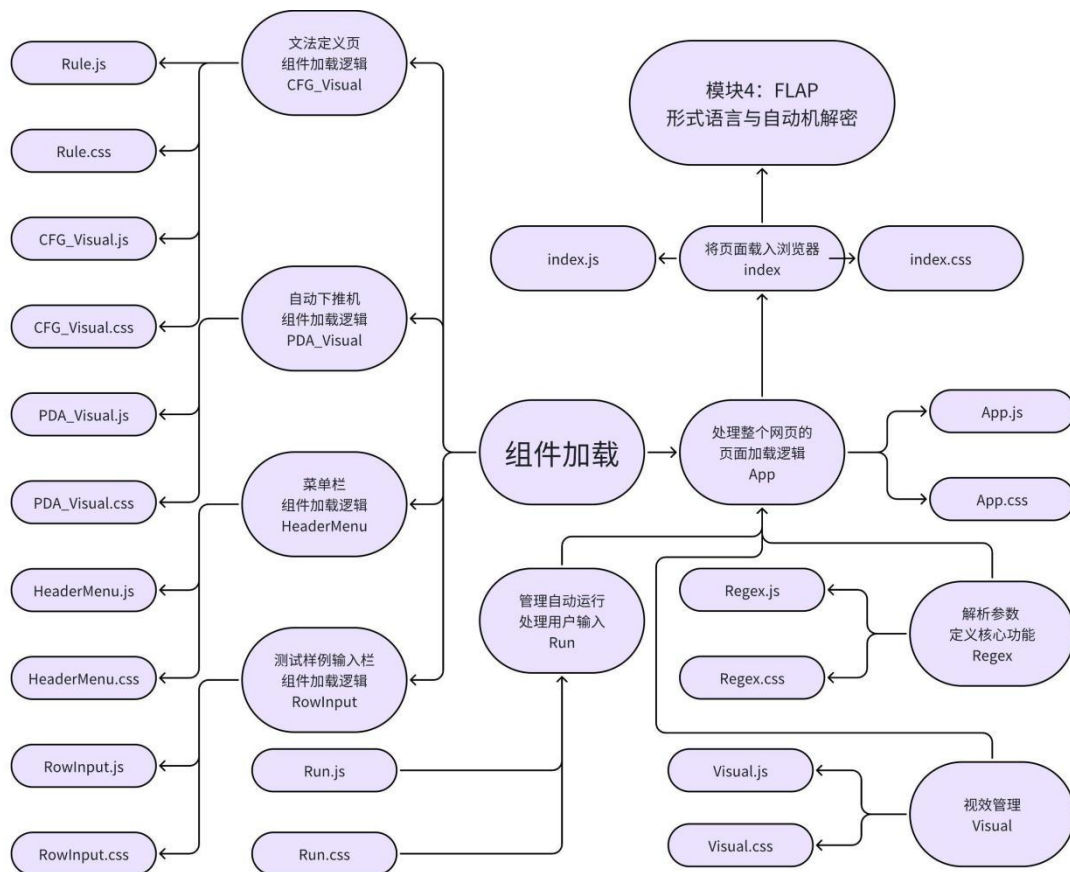
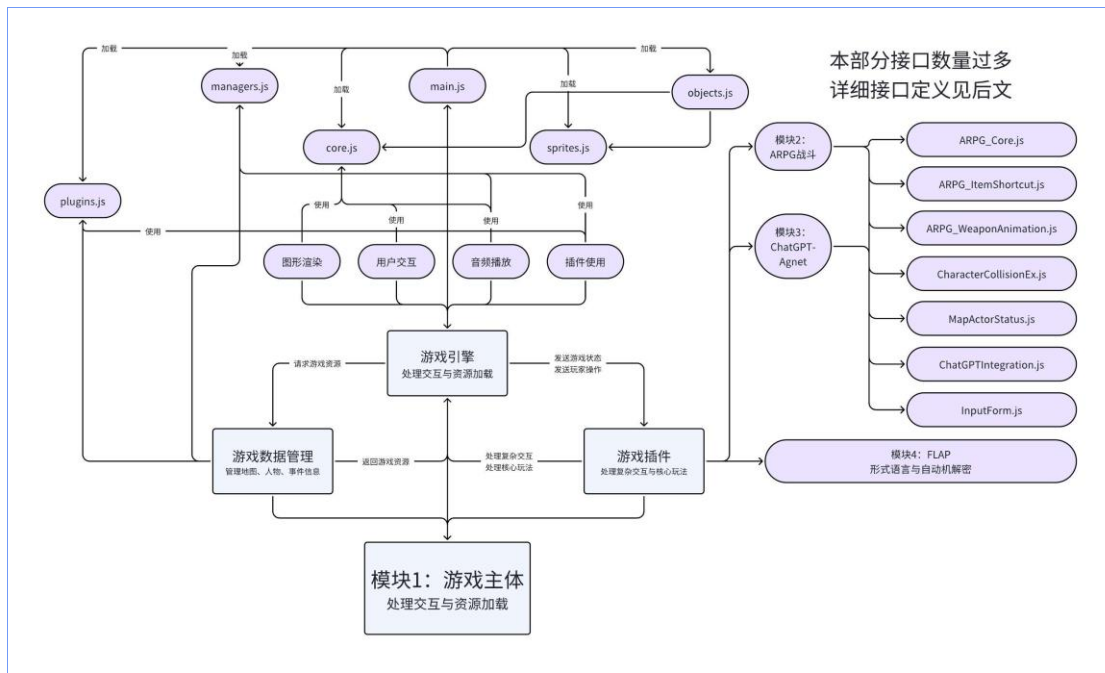
1.6、知识链条



2、总体结构设计

2.1、软件结构

按模块划分本项目，可分为游戏引擎与玩家管理两个大部分，细分为游戏主体、ARPG 战斗插件、Agent 插件、FLAP 插件和玩家管理四个部分。具体软件结构与关系如下：



2.2、设计思想

本软件的基本设计思想为模块化开发。不同模块负责不同的功能。模块间低耦合，不同模块通过定义好的接口通信，便于并行开发，分别维护。

并且，插件模块具有高可扩展性，游戏主体可以通过简单的配置文件修改，达成

复杂的游戏插件添加，而无需修改游戏主体本身。游戏插件也无需跟随游戏主体变化而变化，具有高度可移植性。

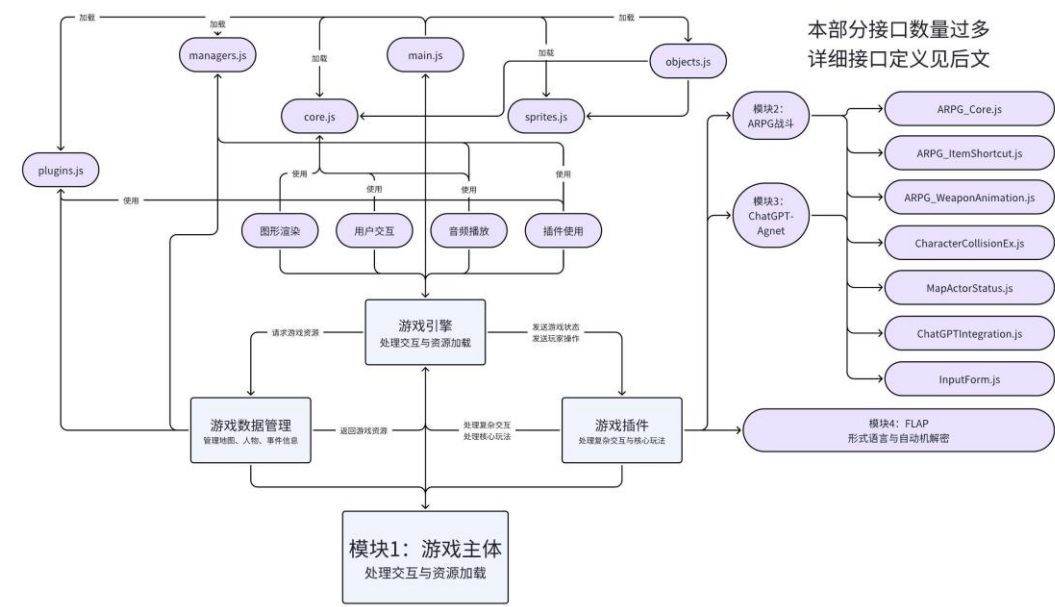
模块间关系呈树状结构，叶节点为 ARPG 战斗插件、Agent 插件、FLAP 插件，分别实现不同的细化功能，并被父节点游戏主体模块统一管理并加载。玩家管理模块作为入口模块，连接至游戏主体模块，并管理玩家信息，可以单独维护管理，减少了后端压力。

3、模块设计

3.1、游戏主体

管理游戏数据和全局信息，负责数据的加载、保存以及错误处理。

设计图



3.2、ARPG 战斗插件

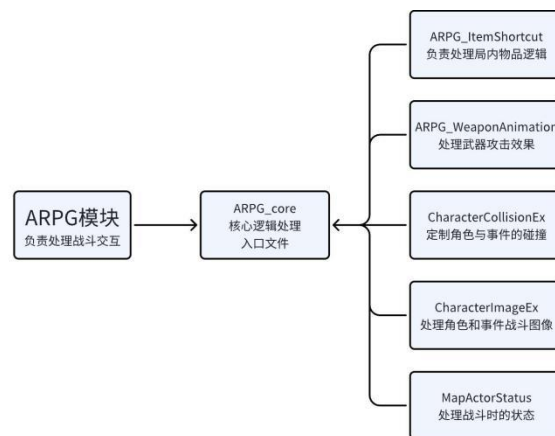
该 ARPG 功能模块主要用于管理游戏中的动作角色扮演（ARPG）元素，通过一系列插件命令，实现角色行为、事件动态生成、技能使用、碰撞检测等功能。具体功能包括：

- 模式切换：启动或结束 ARPG 模式。
- 动态事件生成：在特定位置生成新的动态事件。
- 角色位置获取：获取角色的当前位置或中心位置。
- 角度与距离计算：计算两个角色之间的角度和距离。

- 屏幕内检测：检查角色是否在屏幕内。
- 移动检测：检测角色是否移动。
- 敌人设置：设置事件为敌人，并配置其血量条等属性。
- 血量条显示切换：显示或隐藏角色的血量条。
- 场景对象设置：设置事件为场景对象。
- 技能与物品使用：角色使用技能或物品。
- 技能效果应用：应用技能效果，或测试技能效果是否生效。
- 技能对象生成：生成技能对象，并设置其位置。
- 命中盒设置与检测：设置角色的命中盒，并进行命中检测。
- 角色属性获取与设置：获取或设置角色的特定属性和参数。

击退与动作等待：处理角色击退效果和动作等待。

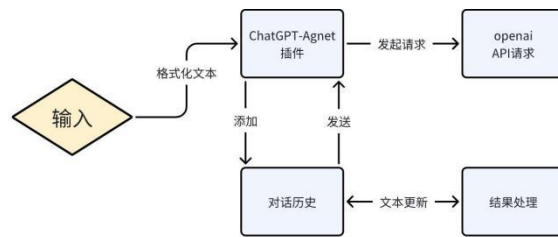
设计图：



3.3、ChatGPT-Agent 插件

该模块是一个集成了 OpenAI 的 ChatGPT API 的 RPG Maker 插件。它的主要功能是将玩家输入的信息发送到 ChatGPT，并将生成的响应返回给游戏以实现动态对话。

设计图：

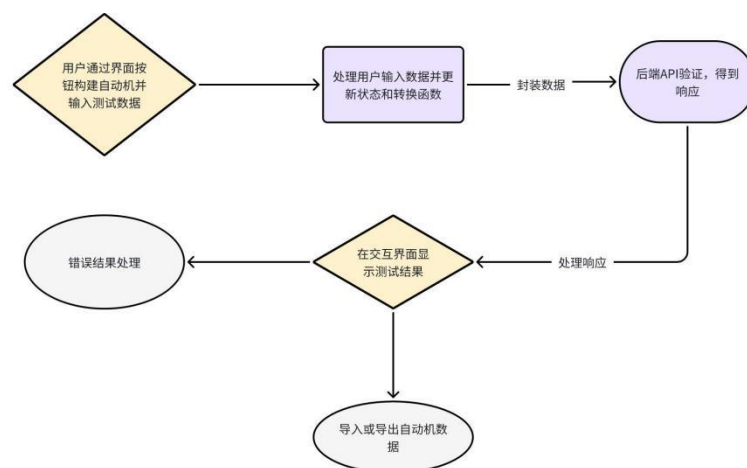


3.4、FLAP 插件

FLAP 是一个自动机可视化模块，它主要负责将用户输入的自动机状态和转换关系可视化，提供友好的界面，帮助用户直观理解自动机的运行和状态转移。它的功能包括：

- 绘制状态图：根据用户输入的状态和转换关系，动态绘制自动机状态图。
- 编辑状态和转换：允许用户通过点击和拖拽操作添加、删除或修改状态和转换关系。
- 设置初始和终止状态：用户可以通过点击操作设置某个状态为初始状态或接受状态。

设计图：

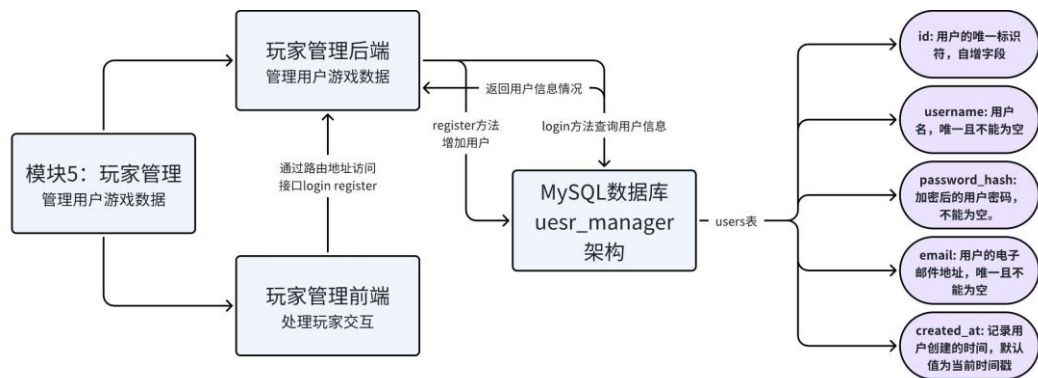


3.5、玩家管理

玩家管理模块的业务功能：

- 注册用户：处理新用户的注册请求，将用户信息保存到数据库中，并加密用户密码。
- 登录用户：处理用户登录请求，验证用户信息并根据验证结果进行相应处理（成功时重定向到游戏页面，失败时返回错误消息）。

设计图：



4、接口设计

4.1 外部接口

描述需要访问的外部接口的类型、控制方式。

4.1.1 OpenAI Chat Completion API 接口

请求方式：通过 POST 请求向远程服务器提交附带请求数据，以生成对话的回复。

URL: <https://api.openai.com/v1/chat/completions>

接口的接入方式：

- 1、准备请求消息：messages 数组包含多个对话条目，构建一个对话上下文，包括一个系统消息（system），描述世界设定和当前剧情，用户消息（user），以及从对话历史中提取的对话条目。
- 2、请求负载（Payload）：payload 是一个包含模型名称（model）和消息（messages）的对象。
- 3、发送请求：使用 fetch 发送 POST 请求到 OpenAI 的聊天完成端点。
请求头（headers）中设置内容类型为 application/json，并包含认证所需的 Authorization 字段（携带 API 密钥 apiKey）。
- 4、处理响应：响应解析为 JSON 格式的数据。从响应中提取生成的回复（如果有），并将其添加到对话历史中。
- 5、处理对话：将相应得到的回复储存到游戏变量中。

4.1.2 RFLAP API 接口

请求方式：根据条件 master_context.PDA 决定请求 URL，然后使用构造好的 JSON 向指定的 URL 发起 POST 请求，以判断用户构造的自动机是否能满足要求。

URL: <https://rflap.acmuic.app/pda> 或 <https://rflap.acmuic.app/automata>

接口的接入方式：

准备 URL：

根据条件 master_context.PDA 决定的，分别指向不同的 Rust API 端点。

数据包处理：

如果 packet_to_misha_the_microsoft_engineer.PDA 为真，需要对数据包进行处理，删除特定的字段并使数据包标准化。

构建请求对象：

构建一个 POST 请求对象，包括请求方法、跨域模式、请求头和请求体（包含 JSON 格式的

数据包)。

发送请求:

使用 `fetch` 发送请求, 并捕获可能的异常。

处理响应与错误处理:

处理 API 响应, 显示测试或考试结果。

4.2 内部接口

描述与其他模块及子系统的接口。

4.2.1 用户注册接口

功能: 处理用户注册请求, 将用户信息插入数据库。

文件: `app.py`

格式: `def register():`

参数: 无

全局变量: `db_config`

局部变量: `data, username, password, email, password_hash, connection, cursor`

返回值: JSON 格式的响应消息

使用约束: POST 请求, Content-Type 为 `application/json`。

4.2.2 用户登录接口

功能: 处理用户登录请求, 验证用户信息。

文件: `app.py`

格式: `def login():`

参数: 无

全局变量: `db_config`

局部变量: `data, username, password, connection, cursor, user`

返回值: JSON 格式的响应消息

使用约束: POST 请求, Content-Type 为 `application/json`。

5、其他设计

5.1、并行处理

并发用户支持: 采用多线程或异步编程技术来处理多个用户的请求, 确保系统能同时处理来自不同用户的输入和反馈。

资源分配和负载均衡: 使用负载均衡技术分配用户请求到不同的服务器实例, 优化资源使用并减少响应时间。

5.2、安全设计

认证和授权: 实现用户登录的认证机制, 确保用户数据的安全。利用 OAuth 或 JWT 等技术进行用户会话管理。

数据加密: 传输中的数据使用 TLS 加密, 数据库存储的敏感信息如用户信息等采用 AES 加密技术。

错误和异常处理: 系统设计要能够妥善处理各类错误和异常, 防止系统崩溃并保护用户数据不被非法访问。

5.3、内存管理

状态图和自动机的内存优化: 采用高效的数据结构存储状态图和自动机模型, 优化内存使用和计算效率。

缓存机制: 对频繁访问的数据如题库和用户成绩进行缓存, 减少数据库访问次数, 提高响应速度。

内存泄露监控: 定期检查和修正可能的内存泄漏问题, 确保系统的稳定运行。